

# Claude Code + GitHub 기반 스크립트 관리 운영 가이드

모델 사용 기본 지침 · 저장소 구조 · 작업 루프 · 자동 검증 · 보안

**요약:** 개발 도구를 "대화창에서 파일 주고받기"에서 **Claude Code(프로젝트 폴더 안에서 직접 수정·검증) + GitHub(이력·자동검증)**로 전환한다. 모델은 작업 성격에 따라 **Sonnet 기본, Haiku 단순작업, Opus 설계·판단**으로 나눠 쓰며, 이 정책은 CLAUDE.md에 기록되어 Claude Code가 매 세션 자동으로 따른다. 동봉 파일 6종(CLAUDE.md, .gitignore, verify.yml, 에이전트 2종, setup\_repo.ps1)을 프로젝트 폴더에 넣고 §7 절차대로 초기화하면 된다.

## 1. 모델 사용 기본 지침

### 1.1 전제 — 어디서 모델이 결정되는가

현재 사용 중인 claude.ai 대화창에서는 **모델 선택권이 사용자에게 있으며**, 대화 상대(Claude)가 스스로 하위 모델로 전환할 수 없다. 반면 Claude Code에서는 세션 중 `/model` 명령으로 전환할 수 있고, 서버에이전트 정의 파일에 `model` 필드를 지정해 작업별로 다른 모델을 자동 배정할 수 있다. 따라서 "판단하여 하위 모델 사용"이라는 요구는 **Claude Code 환경에서 CLAUDE.md 정책 + 서버 에이전트로 구현**하는 것이 실질적 해법이다.

### 1.2 작업별 모델 배정 원칙

| 모델           | 배정 작업                                                                          | 근거                                      |
|--------------|--------------------------------------------------------------------------------|-----------------------------------------|
| Haiku        | 포맷·이름 변경, 주석·docstring 보강, 반복 보일러플레이트, 검증 스크립트 실행·집계 보고                        | 판단이 거의 없는 기계적 작업. 속도·비용 최적              |
| Sonnet (기본)  | 기능 구현, 리팩터링, 버그 수정, 테스트 작성, 문서화, 기존 전략 확장                                      | 코딩 품질과 비용의 균형점. 대부분의 일상 작업              |
| Opus 계열 (상위) | 새 전략 클래스 설계(planner_bridge/dam), 알고리즘·수치 안정성 판단, 좌표계·투영 문제, 난해한 디버깅, 특허 청구항 논리 | 실수 비용이 큰 판단 작업. 설계안만 받고 구현은 Sonnet에 넘긴다 |

운용 요령: 세션은 Sonnet으로 시작하고, 설계 판단이 필요할 때만 `architect` 서버에이전트(Opus)에 설계안을 요청한다. 코드 수정 후에는 `verifier` 서버에이전트(Haiku)가 검증 4종을 돌려 집계만 보고한다. 이렇게 하면 상위 모델 토큰이 "생각하는 부분"에만 쓰인다.

## 2. Claude Code 설치와 시작

| 항목   | 내용                                                                                                                                                                                                                                                   |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 설치   | 공식 문서( <a href="https://docs.claude.com/en/docs/claude-code/overview">https://docs.claude.com/en/docs/claude-code/overview</a> )의 최신 설치 방법을 따른다. npm 패키지(@anthropic-ai/claude-code) 방식은 Node.js가 필요하며, 네이티브 설치기가 제공되는 플랫폼도 있으므로 문서에서 현재 권장 방식을 확인한다. |
| 인증   | claude.ai 구독 계정 로그인 또는 API 키. 회사 API 키는 §6 보안 원칙에 따라 환경변수로만 보관                                                                                                                                                                                       |
| 실행   | 프로젝트 폴더에서 <code>claude</code> 실행 → 대화형 세션. VS Code 확장으로도 연동 가능                                                                                                                                                                                       |
| 첫 세션 | <code>/init</code> 은 CLAUDE.md를 자동 생성하지만, 본 프로젝트는 동봉 CLAUDE.md를 사용하므로 실행하지 않는다. <code>/model</code> 로 현재 모델 확인                                                                                                                                       |

설치 명령·요구 버전 등 세부 사항은 변경이 잦으므로 이 문서에 고정하지 않는다. 항상 위 공식 문서 URL을 기준으로 한다.

### 3. 저장소 구조

|                                                  |                                         |
|--------------------------------------------------|-----------------------------------------|
| C:\Work\FacilityPath\                            | ← Git 저장소 루트 = 가상환경 루트                  |
| ├─ CLAUDE.md                                     | Claude Code 프로젝트 지침 (모델 정책·규칙)          |
| ├─ .claude/agents/                               |                                         |
| │ └─ verifier.md                                 | 검증 실행 전용 (Haiku)                        |
| │ └─ architect.md                                | 설계 검토 전용 (Opus)                         |
| ├─ .github/workflows/verify.yml                  | push/PR 시 자동 검증 (Windows, Py 3.12+3.14) |
| ├─ .gitignore                                    | .venv·산출물·대용량 데이터 제외                    |
| ├─ facilitypath.py                               | 통합 진입점 (generate / adapt)               |
| ├─ io_model.py io_flightpath.py                  | [1] 입력                                  |
| ├─ surface_analyzer.py                           | [2] 대상면 분석                              |
| ├─ camera.py tilt_solver.py                      | [3] 촬영·집벌                               |
| ├─ planner_base.py planner_slope.py              | [4] 경로 생성                               |
| ├─ path_adapter.py                               | [4'] 기존 경로 적응                           |
| ├─ kml_writer.py                                 | [6] 출력                                  |
| ├─ verify_*.py (4종)                              | 자동 검증                                   |
| ├─ requirements*.txt env_check.py setup_env*.ps1 | 환경                                      |
| ├─ README.md                                     | 사용법                                     |
| ├─ docs/                                         | 개발계획서·기술검토·가이드 PDF (커밋 대상)              |
| └─ data/ output/ .venv/                          | 커밋 제외 (.gitignore)                      |

#### 3.1 CLAUDE.md의 역할

Claude Code는 세션 시작 시 저장소의 CLAUDE.md를 읽어 규칙으로 삼는다. 동봉 파일에는 ① 처리 순서(6단계 불변) ② 모델 사용 정책 ③ 환경(3.14, vtk 상한) ④ 필수 규칙(검증 4종 통과 후 커밋, 인코딩, 커밋 메시지 형식) ⑤ 금지 사항이 담겨 있다. 규칙이 바뀌면 이 파일을 고치고 커밋한다 — 대화로 매번 설명할 필요가 없어진다.

## 4. GitHub 워크플로우

### 4.1 브랜치 전략 (소규모 팀 기준, 단순하게)

| 브랜치                   | 용도                                          |
|-----------------------|---------------------------------------------|
| main                  | 검증 통과 버전만. 현장 PC에는 main만 배포. 릴리스 시 태그(v1.1) |
| develop               | 통합 작업. 기능 브랜치가 여기로 합류                       |
| feat/planner-bridge 등 | 기능 단위 작업. 완료 후 develop으로 PR                 |

### 4.2 커밋·PR 규칙

- 커밋 메시지: [M단계] 요약 — 예 [M3] planner\_bridge 파사드 전략 추가, [M4] WPML 열거값 Pilot 2 대조 반영
- PR 생성 시 verify.yml이 Windows에서 Python 3.12·3.14 두 환경으로 검증 4종을 자동 실행한다. **전부 통과해야 병합**(GitHub 저장소 설정 → Branch protection → Require status checks)
- 산출물(KML/KMZ/PNG/CSV)과 대용량 입력(OBJ/LAS/DSM)은 커밋하지 않는다. 실데이터가 필요하면 Git LFS 또는 사내 저장소에 두고 경로만 문서화

### 4.3 릴리스

```
git checkout main && git merge develop
git tag -a v1.1 -m "FacilityPath v1.1 - DSM/기존경로 입력 추가, 검증 37/37"
git push origin main --tags          # 현장 PC: git pull 후 verify 4종 재실행
```

## 5. 일상 작업 루프

대화창 방식 5단계(생성→다운로드→이동→실행→로그 붙여넣기)가 아래 1단계로 줄어든다.

```
PS C:\Work\FacilityPath> .\venv\Scripts\Activate.ps1
PS C:\Work\FacilityPath> git checkout -b feat/planner-bridge develop
PS C:\Work\FacilityPath> claude

> /model                                # Sonnet 확인
> architect 에이전트로 planner_bridge.py 설계안을 먼저 받아줘.
   교량 측면 파사드 + 상판 연직 2세트, planner_base.design_lines 계약 준수.
   (→ Opus가 설계안 제시. 검토 후)
> 설계안대로 planner_bridge.py를 구현하고 verify_planner_bridge.py를 추가해줘.
   (→ Sonnet 구현)
> verifier 에이전트로 검증 4종 + 신규 1종 실행해서 집계 보고해줘.
   (→ Haiku 실행·보고. 전부 PASS 확인 후)
> 커밋해줘. 메시지는 [M3] 규칙으로.
> /exit
PS> git push -u origin feat/planner-bridge    # → GitHub에서 PR → Actions 자동 검증 → 병합
```

**핵심 습관 세 가지:** ① 세션마다 브랜치를 판다 ② 커밋 전 verifier를 돌린다 ③ 사람이 검토해야 할 판단(설계·좌표계·안전 파라미터)은 Claude에게 "결정"이 아니라 "설계안·근거"를 요청하고 직접 승인한다.

## 6. 보안·데이터 원칙

| 항목        | 원칙                                                                                          |
|-----------|---------------------------------------------------------------------------------------------|
| API 키·계정  | 코드·CLAUDE.md·커밋에 절대 넣지 않는다. 환경변수 또는 Claude Code 로그인만 사용                                     |
| 저장소 공개 범위 | Private. 특허 출원(10-2026-0072029) 관련 알고리즘이 포함되므로 공개 저장소 금지                                    |
| 현장 데이터    | 교량·사면 3D 모델과 좌표는 발주처 자료 — 저장소에 올리지 않고 .gitignore로 차단(동봉). 합성 데이터로만 검증                       |
| 도구 권한     | Claude Code의 파일 삭제·git push 등 파괴적 작업은 승인 프롬프트를 끄지 않는다(기본값 유지). 자동 승인은 Read/Grep 수준으로 한정     |
| 개인 설정     | .claude/settings.local.json 은 개인용 — .gitignore로 제외(동봉). 팀 공통 설정만 .claude/settings.json 에 커밋 |

## 7. 초기 세팅 절차 (1회)

1. Git 설치(<https://git-scm.com>), 선택: GitHub CLI(gh). GitHub 계정·조직(droneid) 준비
2. 동봉 6종을 C:\Work\FacilityPath 에 배치: CLAUDE.md, .gitignore, .github/workflows/verify.yml, .claude/agents/verifier.md, .claude/agents/architect.md, setup\_repo.ps1
3. 기존 PDF 문서(개발계획서·기술검토·가이드)는 docs/ 폴더로 이동
4. .\setup\_repo.ps1 실행 → git init(main) + autocrlf 설정 + 첫 커밋
5. GitHub에 **Private** 저장소 생성 후 push (스크립트 말미 안내 A 또는 B)
6. GitHub → Settings → Branches → main 보호 규칙: "Require status checks: verify" 설정
7. Claude Code 설치·로그인 → 프로젝트 폴더에서 claude → /model 확인 → 첫 작업으로 "verifier 에이전트로 검증 4종 실행" 요청해 환경을 확인

## 8. 기대 효과

- **파일 전달 손실 제거:** 지난주 "옛 파일이 돌아갔는지" 혼선은 git log 한 줄로 판별된다
- **환경 재현:** Actions가 3.12·3.14 양쪽에서 매 push마다 검증 → 3.14 적합성이 지속 보증된다
- **비용 통제:** 상위 모델은 설계 판단에만, 검증·정리는 Haiku — 토큰 사용이 작업 가치에 비례한다
- **특허·문서 연계:** 커밋 이력이 "언제 어떤 알고리즘이 구현됐는지"의 증빙이 되어 출원·보고서 작성에 활용된다

작성: (주)드론아이디 AI연구소 · QRFID 디지털트윈 플랫폼 사업부 | Claude Code 설치·명령 세부는 공식 문서([docs.claude.com/en/docs/claude-code](https://docs.claude.com/en/docs/claude-code)) 기준으로 수시 확인 | 동봉: CLAUDE.md, .gitignore, verify.yml, agents 2종, setup\_repo.ps1